

Possible ideas.
How do we accomplish?

All achievable inside the TeamSite constraint. All additive.

The candidates

Skim now. We'll walk through each on the following slides.

1 GitHub as source of truth.

Component code (JS, CSS, XSL, XML) lives in one clean, accurate, documented repo. TeamSite stays the deploy target.

2 Local npm build that validates the repo.

One command lints and checks every component's JS, CSS, XSL and XML before anything goes up.

3 An rbccm-* component library.

Every Block Element Modifier (BEM) named rbccm-* component with all its variations. A local npm build produces a dev page to check the output is what's expected: variations, FPO content, etc.

4 Consolidated CSS + JS bundling.

One main.css and one main.js instead of 12+ standalone files in the head.

5 Localized page preview.

Full pages previewed locally from the #3 component library, matching TeamSite's runtime. Anyone can iterate offline.

6 No reliance on Bootstrap.

Our design system, 1:1 with the Figma expectation. Every component styled in its own SCSS instead of loading a whole library for utility classes. Atomic design principles, 100% accurate.



GitHub as source of truth.

Component code (JS, CSS, XSL, XML) lives in one clean, accurate, documented repo. TeamSite stays the deploy target.

WHAT IT FIXES	History, rollback and review exist today, but loosely (TeamSite versions, local copies, memory). The repo gives one clean place to see what changed, when and why.
COST TO TRY	A repo and a workflow agreement. Still no CI/CD into TeamSite, but code is checked in the repo before anything is uploaded, so "what happened?" has an answer.
TEAMSITE IMPACT	Zero. Deploys stay manual for now, but the repo is the record of what should be live.



Local npm build that validates the repo.

One command lints and checks every component's JS, CSS, XSL and XML before anything goes up.

WHAT IT FIXES	Broken XSL, typos and bad markup found on the live site instead of before upload.
COST TO TRY	One-time script to write. Runs on any dev machine.
TEAMSITE IMPACT	Zero. It only checks the files; uploading to TeamSite is unchanged.

3

An rbccm-* component library.

Every Block Element Modifier (BEM) named rbccm- component with all its variations. A local npm build produces a dev page to check the output is what's expected: variations, FPO content, etc.*

WHAT IT FIXES	The three-carousels pattern. Marketing asking for what already exists. Surprises after publishing.
COST TO TRY	Effort TBD. Builds on the component folders and local tests that already exist.
TEAMSITE IMPACT	Zero. Runs locally, outside TeamSite.



Consolidated CSS + JS bundling.

One main.css and one main.js instead of 12+ standalone files in the head.

WHAT IT FIXES	Asset bloat. Every page paying the tax of every other page's dependencies.
COST TO TRY	Depends on #2 being in place first. Then it's a build-config change.
TEAMSITE IMPACT	Zero. TeamSite serves whatever files the build outputs.



Localized page preview.

Full pages previewed locally from the #3 component library, matching TeamSite's runtime. Anyone can iterate offline.

WHAT IT FIXES	Edits in prod.
COST TO TRY	Comes out of #3: page templates that assemble library components.
TEAMSITE IMPACT	Zero. Runs locally. Only the final files touch TeamSite.



No reliance on Bootstrap.

Our design system, 1:1 with the Figma expectation. Every component styled in its own SCSS instead of loading a whole library for utility classes. Atomic design principles, 100% accurate.

WHAT IT FIXES	Every page loading all of Bootstrap for a handful of utilities. Fighting framework defaults to match the design.
COST TO TRY	In progress now. New components already ship their own CSS; older pages move over as they're rebuilt. Pairs with #4.
TEAMSITE IMPACT	Zero. Future-proofs the rendered output: if the CMS changes, only the templates (PHP, React, etc.) need updating, not the styles.